Livox Mid-360 Emulator

Goal

Build a lightweight daemon that runs on a Raspberry Pi (or any Linux box) and emulates the Livox Mid-360 wire protocol so convincingly that **stock, unmodified livox_ros_driver2 discovers it and publishes point clouds as if a real sensor were attached.** Downstream SLAM stacks should not be able to tell the difference.

Why the Mid-360 ?

- **Unitree/G1 Infrastructure + It's the one people actually deploy for budget SLAM**
- **Best-supported target** in the current stack: Livox-SDK2 and **livox_ros_driver2** both treat Mid-360 (and HAP) as first-class.
- **Highest reuse value** for our work, emulating the sensor everything downstream expects means the emulator is useful across bench testing, HIL, eval, testing, and dataset generation.

Protocol

- UDP, So, Pi's ethernet port will suffice

What we ll need:

Component	Status
Livox-SDK2 (host receive + device control)	**Open, MIT license**
livox_ros_driver2 (ROS/ROS2 wrapper)	**Open**
Device **firmware**	Closed/We might not need this
The **communication protocol spec**	Available

Okay, so **we have both the specification and a reference implementation of the counterparty.** The published protocol tells us exactly what bytes to emit; the open SDK source shows us how the host will parse them. The sensor's own firmware is closed, but that is irrelevant, we are not reproducing how the sensor works internally, only what it says on the wire.

What the spec already gives us

- **Ports (lidar side):** cmd 56100, push_msg 56200, point 56300, IMU 56400, log 56500. Host side: 56101 / 56201 / 56301 / 56401 / 56501.
- **Device type index:** 8 (MID360).
- **Control commands, documented by ID:** 0x0000 discovery-by-broadcast, 0x0100 parameter configuration, 0x0101 parameter inquire, 0x0102 LiDAR information push, plus reboot / factory-reset / set-GPS-timestamp, log, and firmware-upgrade commands.

- **Frame formats and CRC algorithm** for both the control and data planes.
- **Point payload:** offset_time, x/y/z (float, meters), reflectivity (0–255), tag, line.
- **Time sync options:** PTP (IEEE 1588v2.0), gPTP, and GPS (PPS + GPRMC).

What we will build (two planes + pluggable input)

1. **Control plane** — bind the command port and implement the documented handshake: answer 0x0000 discovery, serve 0x0101 inquire, push 0x0102 device info, accept 0x0100 configuration. Nothing streams until the handshake completes, so this comes first — but it is implemented to spec, not guessed.
2. **Data plane** — once handshaked, push Cartesian point packets (~200k pts/s) and a ~200 Hz IMU stream to the host ports.
3. **Pluggable input layer** -> the point source sits behind a fixed protocol layer, so it's swappable:
    - **Replay**: stream recorded .lvx2 / rosbag data (Only 1 in MVP - Replay)
    - **Synthetic:** generate the non-repetitive rosette pattern procedurally
    - **Live transducer:** take a real sensor (e.g. Ouster OS1) plugged into the Pi, mask it through the petal trajectory, re-emit as Mid-360

Validation strategy

The pass/fail judge is stock livox_ros_driver2, unmodified: if it discovers the daemon and publishes CustomMsg, we have succeeded. A reference packet capture from a physical Mid-360 is a useful cross-check for timing realism, but is no longer needed to derive the protocol.

Key risks (Inferred - We might find more when we start implementation)

- **Timing fidelity (PTP/PPS)**: it's the most prominent risk. The Pi's weak hardware timestamping adds jitter that a tightly coupled SLAM system might feel.
- **Packet cadence under load**: the 200 Hz IMU and point streams must stay steady; pin the sender and use a monotonic timer. Bandwidth is trivial for a Pi; steadiness is not automatic.
- **Spec-vs-implementation gaps**: where the published protocol is ambiguous, sdk_core is the tiebreaker, since the driver's actual parsing behavior is what we must satisfy.